



E4 Educator v2.0

# Introduction to the ESP32

*Why this chip, why this board, why this approach**Walark Electronics · Fundrum Engineering · May 2026*

This document is for the person who picks up the E4 Educator binder and asks a reasonable question before committing any time to it: what am I actually learning, and why this particular chip?

If you are an experienced Arduino or Raspberry Pi user, a teacher looking for a structured embedded systems curriculum, or a technically curious person who has outgrown blinking LEDs and wants to build real networked instruments — this is written for you.

## 1 What is the ESP32?

The ESP32 is a microcontroller — a single chip that contains a processor, RAM, flash memory, and a large set of hardware peripherals, all designed to run your code directly with no operating system in between. It was designed by Espressif Systems in Shanghai, first released in 2016, and has since become one of the most widely used microcontrollers in the world for connected devices.

The specific variant used in the E4 Educator is the ESP32-D0WDQ6, mounted on a standard DevKit V1 module. Its headline specifications are modest by desktop computer standards, but they are extraordinary for an embedded chip that costs a few dollars:

| Specification | Detail                                                                                                                    |
|---------------|---------------------------------------------------------------------------------------------------------------------------|
| Processor     | Dual-core Xtensa LX6, up to 240 MHz. Both cores programmable.                                                             |
| RAM           | 520 KB SRAM on-chip. Sufficient for sprite buffers, FFT arrays, JSON documents, and network stacks simultaneously.        |
| Flash         | 4 MB on the DevKit module. Partitioned between firmware, filesystem (LittleFS), and OTA update slots.                     |
| WiFi          | 802.11 b/g/n 2.4 GHz. Built into the chip. Station, access point, and simultaneous AP+STA modes.                          |
| Bluetooth     | Classic Bluetooth and BLE 4.2. Not used in this curriculum but available for extension projects.                          |
| GPIO          | 34 programmable pins. Multiple functions available per pin: digital I/O, ADC, DAC, PWM, I2C, SPI, UART, touch sense, I2S. |
| ADC           | 12-bit analogue-to-digital converter on ADC1 (8 channels). ADC2 shares with WiFi — use ADC1 when WiFi is active.          |
| DAC           | Two 8-bit digital-to-analogue outputs (GPIO 25 and 26). True analogue voltage output — no filtering required for audio.   |
| I2S           | Two I2S buses. Used in this curriculum for the ICS43434 MEMS digital microphone and as a path to digital audio output.    |
| SPI           | Four SPI buses. VSPI used for TFT display, XPT2046 touch controller, and microSD card simultaneously.                     |

|                |                                                                                                                                             |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| I2C            | Two I2C buses. Used for AHT20 temperature/humidity sensor, DS3231 RTC, BMP280 pressure sensor, SSD1306 OLED.                                |
| LEDC           | 16-channel hardware PWM. Used for backlight dimming, piezo tone generation, RGB LED control.                                                |
| ESP-NOW        | Espressif proprietary peer-to-peer wireless protocol. Direct device-to-device communication with no router. Range up to 200m line of sight. |
| Deep sleep     | Current draw as low as 10 $\mu$ A in deep sleep with RTC running. Enables battery-powered sensor nodes with weeks of runtime.               |
| Supply voltage | 3.3 V logic. DevKit module includes USB-C connector with onboard 3.3 V regulator and CP2102 USB-UART bridge.                                |

### The killer feature

The ESP32 is not just a microcontroller with WiFi bolted on. The wireless stack, the dual cores, the peripheral set, and the security engine are all first-class citizens designed together. This is why it has displaced older platforms in commercial products ranging from smart home devices to industrial sensors to musical instruments.

## 2 Why the ESP32 for learning?

There are many microcontroller platforms available, each with genuine strengths. The choice of the ESP32 for the E4 Educator curriculum was deliberate, and it is worth explaining the reasoning so you can evaluate it for your own context.

### 2.1 Compared to Arduino Uno / Nano

The Arduino Uno and Nano use the ATmega328P — an 8-bit processor running at 16 MHz with 2 KB of RAM, 32 KB of flash, and no wireless capability. These constraints are the point for absolute beginners: fewer things that can go wrong. But they become frustrating quickly.

With 2 KB of RAM you cannot have a network stack, a TFT display buffer, a JSON document, and sensor data in memory at the same time. With no WiFi you cannot build connected devices without adding separate modules. With 8-bit integers, floating-point maths is slow software emulation.

The ESP32 removes every one of these constraints while keeping the same Arduino-compatible programming model. A learner who understands the E4 curriculum can read Arduino Uno code without difficulty, and an Arduino user who picks up an ESP32 finds the transition natural. The knowledge transfers in both directions.

### 2.2 Compared to Raspberry Pi

The Raspberry Pi is a single-board computer running Linux — a fundamentally different class of device. It excels at tasks that need a full operating system: web servers, media players, computer vision, machine learning inference. Its GPIO pins are also useful for hardware interaction, but they are managed by the Linux kernel, which introduces timing non-determinism.

The ESP32 runs bare-metal firmware. There is no operating system, no file system daemon, no background processes competing for CPU time. When your code says “toggle this pin every 100 microseconds”, it happens every 100 microseconds. This determinism is essential for audio generation, PWM dimming, I2S microphone capture, and any application where timing precision matters.

For embedded systems learning, the ESP32 teaches the discipline of working within real constraints — managing memory explicitly, avoiding blocking delays, structuring code for hardware events — in a way that a Raspberry Pi, with its Linux safety net, does not.

## 2.3 Compared to other ESP32 boards

The ESP32 ecosystem includes dozens of development boards. The E4 Educator uses the standard ESP32 DevKit V1 module socketed onto a custom carrier board rather than a preconfigured all-in-one module. This choice was intentional.

Preconfigured modules (ESP32-S3 based touch displays, CYD boards, and similar) are excellent for specific applications, but they make tradeoffs invisible. When display, touch, and WiFi all work out of the box with no configuration, the learner does not understand — and cannot later modify — why they work. The E4 Educator exposes every connection: every GPIO assignment is in a build flag, every peripheral is explicitly initialised, every library interaction is visible. When something goes wrong, the learner can diagnose it. That is the whole point.

## 3 Why PlatformIO instead of Arduino IDE?

The Arduino IDE is an excellent entry point. It removes all toolchain complexity and lets a beginner flash their first sketch in minutes. That is its strength, and it does that job well.

PlatformIO is what professional embedded developers actually use. It adds several capabilities that matter as projects grow:

| Feature                    | Why it matters                                                                                                                                                                                                      |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pinned library versions    | espressif32@6.6.0 and TFT_eSPI@2.5.43 are locked in platformio.ini. The project builds identically on any machine, today and in three years. Arduino IDE installs the latest version silently, which breaks things. |
| Build flags                | All GPIO pin assignments live in platformio.ini as -D defines. Change the hardware, change one file. No hunting through source code.                                                                                |
| VS Code integration        | Full IDE features: IntelliSense autocomplete, go-to-definition, error highlighting, integrated git, extensions. The Arduino IDE text editor offers none of these.                                                   |
| Multiple environments      | A single project can target different boards or configurations in the same platformio.ini. Useful for maintaining node and master firmware in the same repository.                                                  |
| LittleFS filesystem upload | Upload the data/ folder to the ESP32 filesystem with one command. The Arduino IDE requires a separate plugin that frequently breaks with IDE updates.                                                               |
| Serial monitor             | Integrated in VS Code, port remembered between sessions, configurable baud rate per project in platformio.ini. No dropdown hunting after every upload.                                                              |

The E4 curriculum uses PlatformIO from Tutorial 1 because the habits formed early persist. A learner who starts with pinned libraries, build flags, and structured project organisation will carry those practices into their own work. A learner who starts with whatever-version-is-installed and magic IDE buttons will spend considerable time later unlearning them.

## 4 What this curriculum builds toward

The E4 Educator curriculum is structured in three tiers. Each tier builds directly on the one before it. By the end, a learner has built five complete, deployable instruments — not demonstration sketches, but firmware that could go into a real product.

| Tier          | Tutorials | What is learned                                                                                                                                                                                                                                              |
|---------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tier 1</b> | T01 - T07 | Platform setup, GPIO, debounced buttons, I2C sensors, OLED display, one-wire sensors, analogue input, non-blocking timing, audio output. No display driver required.                                                                                         |
| <b>Tier 2</b> | T08 - T16 | ILI9341 TFT with sprite pipeline, XPT2046 touch input, SD card logging, NVS persistence, WiFi and MQTT, LittleFS web server, OTA firmware updates, ESP-NOW peer networking, I2S microphone and FFT analysis.                                                 |
| <b>Tier 3</b> | P01 - P05 | Five integrated projects: Environmental Monitor, Data Logger with RTC and Chart.js dashboard, Wireless Sensor Network with deep-sleep nodes, BANDSCOPE audio analyser with dual-core FreeRTOS, Touch UI Controller with page manager and JSON configuration. |

Every project in Tier 3 is specification-driven and version-controlled. The firmware produced is production-quality: no blocking delays in the main loop, proper state machines for WiFi and sensor reads, OTA updates that roll back safely on failure, SD card logging with rotation, and web dashboards served from the device itself.

### What “production-quality” means in this context

Every technique taught in this curriculum is the same technique used in commercial embedded products. Non-blocking state machines instead of `delay()`. Build flags instead of hardcoded constants. Pinned library versions. Three-way version locking across firmware, build system, and documentation. Spec-first development. A learner who completes this curriculum is not learning to tinker — they are learning to build.

## 5 The E4 Educator board

The E4 Educator v2.0 — internally named Enceladus Educator after one of Saturn’s moons — is a purpose-built carrier board for the ESP32 DevKit V1 that brings all the peripherals needed for the full curriculum onto a single PCB.

Rather than an all-in-one integrated module, it uses a socketed ESP32 so the module can be replaced if damaged, updated to a newer variant, or reused in another project after the course is complete. Every peripheral on the board is connected through the standard ESP32 pin header — there is no hidden routing, no proprietary protocol, and nothing that cannot be replicated on a breadboard if needed.

| On-board hardware    | Purpose in curriculum                                                                              |
|----------------------|----------------------------------------------------------------------------------------------------|
| ILI9341 TFT, 320×240 | Tier 2 display tutorials T08–T09. All Tier 3 projects. Landscape orientation with sprite pipeline. |
| XPT2046 touch        | T09 touch input. All Tier 3 projects with interactive UI.                                          |
| microSD slot         | T10 file I/O. P01–P03 data logging. Factory test record.                                           |
| AHT20 (I2C)          | T04 I2C introduction. P01–P02 environmental sensing.                                               |

|                       |                                                             |
|-----------------------|-------------------------------------------------------------|
| ICS43434 MEMS mic     | T16 I2S audio. P04 BANDSCOPE spectrum analyser.             |
| DS18B20 header        | T06 one-wire sensor. P01–P02 probe temperature.             |
| LDR + divider         | T02 and T06 analogue input. P01–P03 light sensing.          |
| RGB LED               | T02 GPIO output. Visual feedback throughout all tutorials.  |
| Piezo buzzer          | T07 TonePlayer.h audio. Alert feedback in P01 and P05.      |
| DAC header (GPIO 25)  | T07 analogue audio output. Sine wave generation.            |
| Two tactile buttons   | T03 Button.h debounce. Navigation throughout all tutorials. |
| I2C expansion headers | P02 DS3231 RTC and BMP280 pressure. User expansion.         |
| BC847 backlight PWM   | T02 LEDC PWM. Brightness control in all display projects.   |

## 6 A note for teachers

The E4 Educator curriculum was designed with structured delivery in mind, but it is not prescriptive about pace or grouping. Each tutorial is self-contained: it lists the hardware required, the skills introduced, and the connection to subsequent work. A student who completes a tutorial and wants to explore further will find extension suggestions at the end of every document.

The three-tier structure maps naturally to course modules. Tier 1 (T01–T07) can stand alone as a seven-session introduction to embedded systems requiring no display hardware. Tier 2 (T08–T16) introduces the full board capability and can form the core of a semester course. Tier 3 projects are suitable for individual or small-group capstone work.

### Assessment opportunities

Each Tier 3 project includes a complete feature specification, commissioning checklist, and version history table — structures familiar from professional practice. These make natural assessment frameworks: can the student flash and commission a working board? Can they modify a feature and update the version record? Can they diagnose a failure from the commissioning checklist?

The factory test firmware (a separate document in this set) provides an objective, automated pass/fail result for each board before it is issued to a student. This eliminates time lost to hardware faults that are mistaken for code errors — a common frustration in workshop settings.

The full source code for every tutorial and project is included in the accompanying zip file. All code is written to the same standard: consistent naming, build flag discipline, no magic numbers, clear comments. Students reading production-quality code from the start develop better intuitions about structure and style than students who learn from sketches written for brevity.

## Ready to begin?

The next document in the binder is the Quick Start Guide. It will have you building and flashing your first E4 firmware in under fifteen minutes.

*Walark Electronics · Fundrum Engineering · Christchurch, New Zealand*